

I . Problem Description

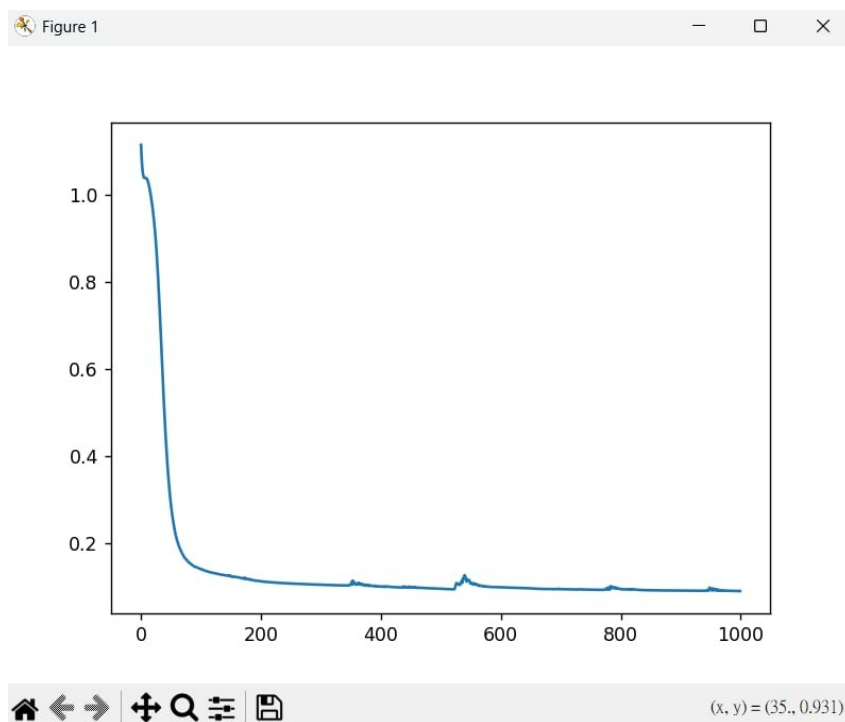
1. Modify the example of multispectral image data classification in the following two aspects. First, change the neural network to a 4-layer structure. You can choose the number of hidden nodes in the network. Second, change the loss function to cross entropy. Test the performance of the new network. Please hand in the printed source code and the test results of your work.

4-layer structure: (example2pytorch.py) (4,16,8,5)

```
model = nn.Sequential(  
    nn.Linear(4, 16),  
    nn.Sigmoid(),  
    nn.Linear(16, 8),  
    nn.Sigmoid(),  
    nn.Linear(8, 5),  
    nn.Sigmoid(),  
    nn.Linear(5, 3)  
)
```

Loss function: cross-entropy, learning rate = 0.02

```
criterion = nn.CrossEntropyLoss()  
  
optimizer = optim.Adam(model.parameters(), lr=0.02)
```



II. Training result

Training Accuracy: 0.9621052742004395

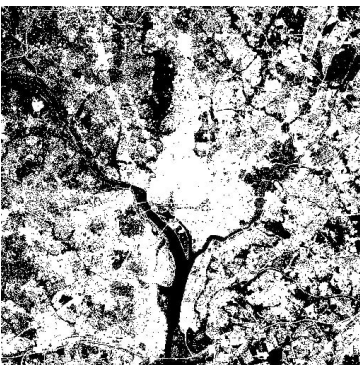
Validation Accuracy: 0.9699525833129883 (no over-fitting)

1. original graph: color.png

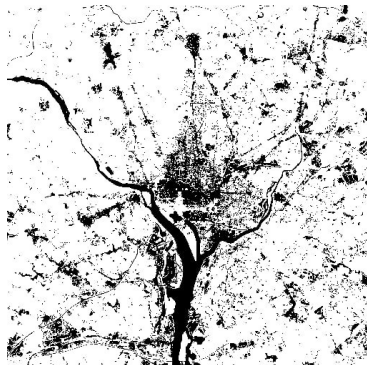


2. training result graph for the white parts:

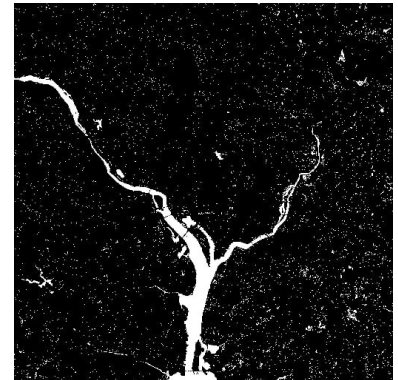
urban.png



veg.png



water.png



III . Data Analysis and source code (tensor: weight and bias)

Environment: C:\AI\venv\Scripts\python.exe C:\AI\example2\example2pytorch.py

Epoch: 0, Loss: 1.1141971349716187

Epoch: 100, Loss: 0.14095599949359894

Epoch: 200, Loss: 0.11277694255113602

Epoch: 300, Loss: 0.10489043593406677

Epoch: 400, Loss: 0.10108840465545654

Epoch: 500, Loss: 0.0957324281334877

Epoch: 600, Loss: 0.09903766959905624

Epoch: 700, Loss: 0.09463688731193542

Epoch: 800, Loss: 0.09561381489038467

Epoch: 900, Loss: 0.09144596755504608

Epoch: 999, Loss: 0.09049345552921295

Training Accuracy: 0.9621052742004395

Validation Accuracy: 0.9699525833129883

Layer (type)	Output Shape	Param #
=====		
Linear-1	[-1, 100, 16]	80
Sigmoid-2	[-1, 100, 16]	0
Linear-3	[-1, 100, 8]	136
Sigmoid-4	[-1, 100, 8]	0
Linear-5	[-1, 100, 5]	45
Sigmoid-6	[-1, 100, 5]	0
Linear-7	[-1, 100, 3]	18

Total params: 279

Trainable params: 279

Non-trainable params: 0

Input size (MB): 0.00

Forward/backward pass size (MB): 0.05

Params size (MB): 0.00

Estimated Total Size (MB): 0.05

first layer(16X4 weight): tensor([[0.3123, 0.3703, -0.0225, 0.2434],
[0.3892, -0.5084, 0.4123, -0.3301],
[0.1093, 0.4051, -0.4958, -0.5541],
[0.3330, -0.7369, -0.6363, 0.4717],
[-0.0960, 0.2115, 0.0808, -0.6814],
[-0.0943, -0.5264, 0.3382, -0.4629],
[0.3634, 0.5370, 1.0856, -0.6096],
[0.5672, 0.0116, 0.2516, -0.9036],
[0.2174, -0.3605, -0.1106, 0.8029],
[-0.0696, 0.4085, -0.5367, -0.0157],
[-0.3573, -0.4661, -1.0753, 0.5100],

[-0.2919, 0.0768, 0.0306, 0.5913],
[0.1251, -0.3740, 0.5322, 0.3407],
[-0.8528, -0.1352, -0.1152, -0.1485],
[-1.2097, -0.0135, 0.3060, 0.6102],
[-0.7968, -0.5822, -0.1151, 0.8332]])

(16 bias): tensor([0.0963, 6.8924, 1.7032, -0.4089, -0.5685, 0.2974, 3.6093, 4.5314,
-0.6379, 7.6665, 4.3968, 0.1784, -5.7913, -0.5039, 2.3364, -0.4597])

second layer(8X16 weight):

tensor([[-6.8726e-02, -1.3066e-01, -9.4443e-01, 2.2723e+00, -1.0370e+00,
-1.9583e-01, -7.4168e-01, -1.9399e+00, 1.2442e+00, -5.1198e+00,
2.6046e+00, 9.5236e-01, 9.9400e-01, 6.5420e-01, 3.6973e-02, 1.6286e+00],
[-8.0141e-01, -5.4658e-01, -1.2250e+00, -1.6035e+00, -8.4763e-01,
1.6479e-01, -2.6325e+00, 1.0231e-01, 1.7932e+00, -1.1936e+00,
-2.3935e+00, 1.4062e+00, 2.2855e+00, 9.9003e-02, -6.3057e-01, -4.9000e-01],
[-2.6819e-01, -7.3483e-01, -1.2815e+00, -1.9028e+00, -1.0297e+00,
2.9902e-01, -2.6852e+00, -5.1595e-01, 1.6707e+00, -1.5641e+00,
-3.1912e+00, 1.6840e+00, 1.9681e+00, 2.2656e-01, -7.1057e-01, -7.3812e-01],
[1.7013e+00, 4.6724e-01, 1.1033e+00, 9.1699e-01, 9.7659e-01,
9.1067e-02, 8.2638e-01, -4.1879e+00, -2.4652e-01, 1.8181e+00,
1.4453e+00, -2.4625e-01, -6.6075e-01, -5.7451e-01, 1.0657e+00, 3.6162e-01],
[-3.8950e-01, 4.4385e+00, -7.0129e-01, 8.1836e-01, 3.2236e-01,
3.0044e-04, 1.1208e+00, 2.1602e+00, -2.4457e-03, 1.8070e+00,
-1.3416e+00, -2.8304e-02, -1.6886e-01, 3.9573e-01, -2.3140e+00, -2.0110e+00],
[-1.8962e-01, 7.3404e-01, 7.0789e-01, -1.3665e+00, 1.2410e+00,
1.1464e-03, 4.3592e-01, 1.1441e+00, -7.9328e-01, 3.8749e+00,
-2.5013e+00, -8.8596e-01, -1.1696e+00, -7.8437e-01, 4.7296e-01, -1.6216e+00],
[-4.8691e-01, -1.7329e+00, -1.3954e+00, -7.2035e-01, -1.3726e+00,
-1.6824e-01, -5.2261e-01, -2.6851e-01, 1.4799e+00, -3.0763e+00,
-3.6104e+00, 1.5594e+00, 2.0944e+00, -7.9272e-02, 3.1553e-01, 8.7114e-01],
[-6.2017e-02, 1.4919e+00, 1.0894e+00, 5.3055e+00, 1.3015e+00,
2.0328e-01, 2.6906e-01, -1.5958e-01, -1.2622e+00, 3.8961e+00,
-5.6201e+00, -7.3005e-01, -1.4036e+00, -8.6762e-01, 1.0503e+00, -2.3116e+00]])

(8 bias): tensor([-0.0794, -0.5028, -0.2376, 1.2596, -0.5819, 0.1775, -0.7626, 0.1979])

third layer: tensor([[-2.2702, 1.1736, 0.7687, -1.7021, 2.1441, 2.3876, 0.6883, 2.9033],
[-1.3101, -2.0787, -2.4587, 1.3934, -0.4293, 0.9963, -2.5465, 1.3427],
[2.3286, 0.9990, 0.9411, 0.2570, -1.7057, -2.1521, 1.2837, -2.3470],
[2.3553, 1.6430, 0.7692, -0.9197, -0.3319, -2.9761, 0.9334, -2.5391],
[1.3846, -2.6689, -2.8188, 1.7786, -1.6116, 0.7298, -2.2107, 0.5084]]) **(5X8 weight)**
(5 bias): tensor([-0.5781, 1.1950, 0.7774, 1.8614, 1.2225])

fourth layer(3X5 weight): tensor([[2.5522, 2.4804, -2.6740, -2.7598, 1.8995],
[3.0573, -2.4628, 1.5666, 1.8571, -2.7523],
[-3.1167, 0.4562, 2.0843, 1.6218, 2.2238]])
(3 bias): tensor([0.0415, 0.3879, -0.1269])

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchsummary import summary

import numpy as np
import matplotlib.pyplot as plt

import cv2

with open('example2data.npy', 'rb') as f:
    img = np.load(f)
    data_train = np.load(f)
    data_test = np.load(f)
f.close()

model = nn.Sequential(
    nn.Linear(4, 16),
    nn.Sigmoid(),
    nn.Linear(16, 8),
    nn.Sigmoid(),
    nn.Linear(8, 5),
    nn.Sigmoid(),
    nn.Linear(5, 3)
)

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(model.parameters(), lr=0.02)

history = []
for epoch in range(1000):
    np.random.shuffle(data_train)
    x_train = torch.tensor(data_train[:,0:4], dtype=torch.float32)
    y_train = torch.tensor(data_train[:,4:7], dtype=torch.float32)
    optimizer.zero_grad()
    y_pred = model(x_train)
    loss = criterion(y_pred, y_train)
    history.append(loss.detach())
    print('Epoch: ' + format(epoch) + ', Loss: ' + format(loss))
    loss.backward()
```

```

optimizer.step()

label_train = torch.max(y_train, dim=1)[1]
label_pred = torch.max(y_pred, dim=1)[1]
acc = (label_train == label_pred)
n_train = float(x_train.shape[0])
print("Training Accuracy: " + format(sum(acc)/n_train))

x_val = torch.tensor(data_test[:,0:4], dtype=torch.float32)
y_pred = model(x_val)
y_val = torch.tensor(data_test[:,4:7], dtype=torch.float32)
label_val = torch.max(y_val, dim=1)[1]
label_pred = torch.max(y_pred, dim=1)[1]
acc = (label_val == label_pred)
n_val = float(x_val.shape[0])
print("Validation Accuracy: " + format(sum(acc)/n_val))

img = np.moveaxis(img, 0, -1)
imgclass = model(torch.tensor(img, dtype=torch.float32))
imgclass = imgclass.detach().numpy()
imgclass = (imgclass > 0.5).astype('uint8') * 255
cv2.imwrite('water.png',imgclass[:, :,0])
cv2.imwrite('urban.png',imgclass[:, :,1])
cv2.imwrite('veg.png',imgclass[:, :,2])
cv2.imwrite('color.png',img[:, :,0:3])
summary(model, (100,4))
for param in model.parameters():
    print(param.data)

plt.plot(history)
plt.show()

```